

Inventory Management SaaS MVP — Master Planning Document

1. Project Vision

Project Name

Inventory Management SaaS MVP

Project Goal

Build a multi-platform inventory management SaaS system that helps businesses:

- Track inventory accurately
- Reduce operational mistakes
- Prevent stockouts
- Improve warehouse organization
- Manage purchases and sales efficiently
- Improve financial visibility
- Support offline-first operations

The system will initially support:

- Supermarkets
- Clothing stores
- Electronics stores
- General retail businesses

The architecture should be extensible for future business-specific modules.

2. Technology Stack

Layer	Technology
Frontend	Flutter
Backend	Supabase
Local Database	sqlite
Authentication	Supabase Auth
Cloud Database	PostgreSQL (Supabase)

Layer	Technology
File Storage	Supabase Storage
Notifications	Local notifications initially (FCM later optional)
Platforms	Mobile + Web + Desktop
Architecture	Clean Architecture
State Management	Bloc/Cubit

3. High-Level System Architecture

```
Flutter App
  ↓
Presentation Layer
  ↓
Domain Layer
  ↓
Data Layer
  ↓
Local Database (sqflite)
  ↓
Sync Engine
  ↓
Supabase Cloud
  ↓
PostgreSQL
```

4. Architecture Principles

Core Principles

- Offline-first
- Multi-tenant SaaS
- Modular design
- Scalable architecture
- Real-time inventory updates
- Inventory movement-based system
- Role-based access control
- Business-type extensibility

5. Multi-Tenant SaaS Design

Concept

Many businesses use the same platform.

Each business has:

- isolated products
- isolated users
- isolated inventory
- isolated warehouses
- isolated reports

Example

Business A

- products
- users
- warehouses

Business B

- products
- users
- warehouses

6. Business Types

Supported Business Types (Initial)

Business Type	Special Features
Supermarket	Barcode-heavy workflow
Clothing Store	Variants (size/color)
Electronics	Serial tracking later
Generic Retail	Core inventory features

7. MVP Scope (IMPORTANT)

Core MVP Features

Authentication

- Login
 - Register
 - Forgot password
 - Session persistence
-

Business Setup

- Create business
 - Select business type
 - Invite employees
-

Product Management

Product Fields

- Product name
 - SKU
 - Barcode (optional initially)
 - Category
 - Cost price
 - Selling price
 - Current stock
 - Minimum stock
 - Unit type
 - Product image
 - Notes
-

Category Management

- Create categories
 - Edit categories
 - Delete categories
 - Product-category relationship
-

Supplier Management

- Create suppliers
 - Edit suppliers
 - Supplier contact info
 - Purchase history
-

Warehouse Management

Initial MVP

- Warehouse name
- Warehouse code
- Basic location support

Future

- Zones
 - Shelves
 - Bins
-

Stock Movement System (CORE FEATURE)

IMPORTANT: Inventory quantity should NOT be edited directly.

Inventory changes through stock movements.

Movement Types

Type	Effect
Purchase	Increase stock
Sale	Decrease stock
Return	Increase/decrease
Transfer	Move between warehouses
Adjustment	Manual correction
Damage/Loss	Decrease stock

Purchase Management

- Create purchase order
 - Receive inventory
 - Update stock automatically
 - Track supplier purchases
-

Sales Management / POS

- Create sale
 - Select products
 - Update inventory automatically
 - Generate receipt/invoice
-

Inventory Adjustments

- Missing inventory
 - Damaged inventory
 - Expired inventory
 - Manual correction reason
-

Dashboard

Metrics

- Total products
 - Inventory value
 - Low stock products
 - Recent movements
 - Sales summary
 - Purchases summary
-

Notifications

- Low stock alerts
 - Inventory warnings
-

Reports

Initial Reports

- Sales report
 - Purchase report
 - Inventory valuation
 - Low stock report
 - Slow-moving inventory
-

Roles & Permissions

Role	Permissions
Owner/Admin	Full access
Warehouse Employee	Stock operations
Cashier	Sales only

8. Future Features (NOT MVP)

Advanced Analytics

- ABC analysis
 - EOQ suggestions
 - Forecasting
 - Profit analytics
 - Demand prediction
-

Advanced Inventory Features

- FIFO costing
 - Batch tracking
 - Expiration tracking
 - Serial numbers
 - Multi-warehouse transfers
 - Warehouse shelves/zones
-

Advanced SaaS Features

- Subscription plans
 - Billing
 - Organization management
 - Audit logs
 - API integrations
-

9. Core Business Workflows

Workflow 1 — Product Purchase

```
Create Purchase Order
  ↓
Receive Products
  ↓
Create Stock Movement (+)
  ↓
Update Inventory
  ↓
Update Dashboard
```

Workflow 2 — Product Sale

```
Create Sale
  ↓
Validate Stock Availability
  ↓
Create Stock Movement (-)
  ↓
Update Inventory
  ↓
Generate Invoice
```

Workflow 3 — Inventory Adjustment

```
Physical Count
  ↓
```

Difference Detected
↓
Create Adjustment
↓
Create Stock Movement
↓
Update Inventory

10. Database Design (Initial Draft)

businesses

Field
id
name
business_type
created_at

users

Field
id
business_id
name
email
role

products

Field
id
business_id

Field
category_id
name
sku
barcode
cost_price
selling_price
minimum_stock
unit_type
image_url
created_at

warehouses

Field
id
business_id
name
code

inventory

Field
id
product_id
warehouse_id
quantity
updated_at

stock_movements

Field
id
product_id
warehouse_id
movement_type
quantity
reference_id
notes
created_by
created_at

suppliers

Field
id
business_id
name
phone
address
notes

purchases

Field
id
business_id
supplier_id
total_amount

Field
status
created_by
created_at

purchase_items

Field
id
purchase_id
product_id
quantity
cost_price

sales

Field
id
business_id
total_amount
created_by
created_at

sale_items

Field
id
sale_id
product_id
quantity

Field

selling_price

11. Why Supabase Is Better For This Project

Relational Architecture Advantage

Inventory systems are highly relational systems.

This project contains relationships like:

- businesses
- products
- warehouses
- suppliers
- sales
- purchases
- stock movements

Using PostgreSQL through Supabase makes these relationships easier to manage compared to NoSQL databases.

Supabase Advantages For This SaaS

- SQL relational modeling
- Easier reporting and analytics
- Better SaaS scalability
- Easier subscription modeling later
- Better support for business workflows
- Row Level Security for multi-tenant SaaS
- Real-time support
- PostgreSQL reliability

Recommended Backend Architecture

```
Flutter
  ↓
sqflite
  ↓
Repository Layer
```

```
↓  
Supabase  
↓  
PostgreSQL
```

12. Offline-First Strategy

Why Offline-First?

Inventory systems must continue working when:

- internet disconnects
- warehouse WiFi fails
- cashier offline

Local-First Flow

```
User Action  
↓  
Save to sqflite  
↓  
Update UI instantly  
↓  
Queue sync operation  
↓  
Sync with Supabase later
```

13. Recommended Flutter Folder Structure

```
lib/  
├── core/  
├── features/  
│   ├── auth/  
│   ├── products/  
│   ├── inventory/  
│   ├── purchases/  
│   ├── sales/  
│   └── suppliers/
```

```
|   | dashboard/  
|   | settings/  
| shared/  
| main.dart
```

14. Clean Architecture Structure

```
feature/  
| data/  
| domain/  
| presentation/
```

data/

- models
- repositories
- datasources

domain/

- entities
- usecases
- repository contracts

presentation/

- screens
 - cubits/blocs
 - widgets
-

15. Important Engineering Decisions

Decision 1

Use stock movements instead of direct quantity editing.

Reason:

- auditability
 - history tracking
 - professional inventory architecture
-

Decision 2

Manual selling price management.

Reason:

- pricing is business decision
 - owner control required
-

Decision 3

Use SKU as internal identifier.

Reason:

- organization
 - searchability
 - categorization
-

Decision 4

Business-type extensibility.

Reason:

- scalable SaaS architecture
 - future customization support
-

16. Suggested Development Order

Phase 1

- Project setup

- Clean architecture setup
 - Supabase integration
 - Authentication
-

Phase 2

- Product management
 - Categories
 - Suppliers
-

Phase 3

- Inventory system
 - Stock movements
 - Warehouses
-

Phase 4

- Purchases
 - Sales
 - Inventory adjustments
-

Phase 5

- Dashboard
 - Reports
 - Notifications
-

Phase 6

- Offline sync optimization
 - Testing
 - Desktop/web polishing
-

17. Current Priority

Your current focus should be:

1. Finalize MVP scope
 2. Finalize database/domain model
 3. Design stock movement architecture
 4. Setup clean architecture Flutter project
 5. Start implementation gradually
-

18. Important Final Advice

Do NOT try to build:

- full ERP system
- all advanced analytics
- all business types deeply

Your goal is:

- operationally realistic MVP
- scalable architecture
- strong portfolio project
- deep business/domain understanding

If your system correctly handles:

- products
- purchases
- sales
- stock movements
- inventory tracking
- basic reporting

then you already built something very valuable.